Bluetooth Slave Mode Experiment

1. BLE Peripheral Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low-power, low-bandwidth scenarios. Compared with Classic Bluetooth, BLE has a faster connection speed (millisecond-level) and lower power consumption (coin-cell batteries can run for months). The ESP32-S3 supports NimBLE (MicroPython's default BLE stack). In slave mode (Peripheral), the ESP32-S3 advertises its presence and waits for a central (another ESP32) to connect for bidirectional data exchange.

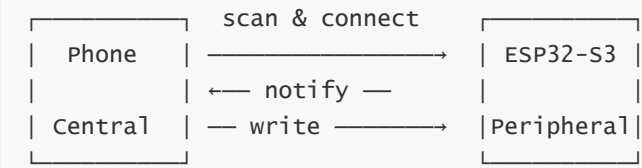
This experiment **implements a BLE Peripheral (slave)**, advertising outward with the device name `"ESP-IDF"`, providing a custom GATT service `0xFFF0`, and supporting notification reporting (0xFFF1) and write reception (0xFFF2).

Typical application scenarios:

Application Type	Example
Phone accessories	Wristbands, heart rate straps, thermometers
Near-field configuration	Phone App provisions the ESP32 via BLE
Sensor nodes	Low-power sensors periodically broadcast data
Door locks/tags	BLE access control, anti-loss trackers

2. Core Concepts

2.1 BLE Slave Role



- **GATT**: Generic Attribute Profile, defining the hierarchical structure of Service -> Characteristic
- **Service**: functional grouping (UUID: 0xFFF0)
- **Characteristic TX**: Notify property, the slave actively pushes data to the central (UUID: 0xFFF1)
- **Characteristic RX**: Write property, the central writes data to the slave (UUID: 0xFFF2)

Operation	Initiator	Target	Acknowledged	Typical Scenario
Notify	Slave -> Central	0xFFF1	No acknowledgment	Sensor data reporting, heartbeat counting
Write	Central -> Slave	0xFFF2	Requires link-layer acknowledgment	Control command delivery, parameter configuration

2.2 Bluetooth Protocol Hierarchy

```
APP layer: your code (send/receive data processing)
  |
GATT layer: Service -> Characteristic (data organization)
  |
GAP layer: advertising + connection management (device discovery)
  |
Link Layer + PHY: radio
```

2.3 Core API

```
import bluetooth
from micropython import const

_IRQ_CENTRAL_CONNECT    = const(1)   # central connected
_IRQ_CENTRAL_DISCONNECT = const(2)   # central disconnected
_IRQ_GATTS_WRITE        = const(3)   # central wrote data

ble = bluetooth.BLE()                 # get BLE instance
ble.active(True)                      # activate BLE
ble.irq(callback)                     # register IRQ callback
ble.gatts_register_services((SVC,))   # register services
ble.gap_advertise(100000, adv_data=...) # start advertising
ble.gatts_notify(conn, handle, data)  # send notification
ble.gatts_read(handle)                # read characteristic value
```

3. Hardware Dependencies

The ESP32-S3 has a built-in BLE module, **no external hardware needed**. Two ESP32-S3 boards are needed for dual-board debugging (one master, one slave).

3.1 ESP32-S3 BLE Hardware Features

Feature	Parameter
Wireless standard	BLE 5.0
Max transmit power	+20 dBm
Receive sensitivity	-97 dBm (1Mbps mode)
Max MTU	251 bytes (256 negotiated as the upper limit in this project)
Antenna	On-board PCB antenna
Advertising interval	Stack default (~100ms)
Protocol stack	NimBLE (Apache open source)

> BLE and WiFi share the 2.4 GHz RF front end; the two can coexist through time-division multiplexing.

---

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  |- BLESlave class initialization:
  |    |- ble.active(True) + irq callback registration
  |    |- gatts_register_services (register TX + RX characteristics)
  |    |- gap_advertise (start advertising, name "ESP-IDF")
  |
  |- while True:
       |- tick(): check whether re-advertising is needed
       |- ble.send(data): push data to the connected central (Notify)
       |- sleep(500ms)

IRQ callbacks:
  |- _IRQ_CENTRAL_CONNECT    -> print "BLE Connected"
  |- _IRQ_CENTRAL_DISCONNECT -> mark that re-advertising is needed
  |- _IRQ_GATTS_WRITE        -> read data + print
```

---

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> BLE_Slave -> ble_slave.py`

5.1 Defining the Service and Characteristics

GATT hierarchy: Service (0xFFF0) contains two characteristics — TX (Notify, slave->central) and RX (Write, central->slave).

```
_SVC_UUID    = bluetooth.UUID(0xFFF0)
_CHR_TX_UUID = bluetooth.UUID(0xFFF1)  # Notify: ESP32 -> central
_CHR_RX_UUID = bluetooth.UUID(0xFFF2)  # Write:  central -> ESP32

_FLAG_READ   = const(0x0002)
_FLAG_NOTIFY = const(0x0010)
_FLAG_WRITE  = const(0x0008)

_SVC = (_SVC_UUID, (
    (_CHR_TX_UUID, _FLAG_READ | _FLAG_NOTIFY),     # TX: readable + notifiable
    (_CHR_RX_UUID, _FLAG_WRITE),                    # RX: writable
))
```

5.2 Initialization and Advertising

`ble.active(True)` -> register the Service -> `gap_advertise` starts advertising. The advertising packet is assembled from three parts: Flags + Service UUID + Device Name.

```
class BLESlave:
    def __init__(self, name="ESP-IDF"):
        self._ble = bluetooth.BLE()
        self._ble.active(True)
        self._ble.irq(self._irq)                      # all events go through the
irq callback

        ((self._tx_handle, self._rx_handle),) = \
            self._ble.gatts_register_services((_SVC,))

        self._name = name
        self._pending_advertise = True

    def _start_advertising(self):
        name_bytes = self._name.encode()
        self._ble.gap_advertise(100_000, adv_data=(
            b'\x02\x01\x06'                           # Flags: LE General
Discoverable
            + b'\x03\x03\xF0\xFF'                     # 16-bit Service UUID:
0xFFF0
            + struct.pack('BB', len(name_bytes) + 1, 0x09) + name_bytes  #
device name
        ))
```

> The advertising packet is assembled from three parts: Flags (general discoverable) + Service UUID (declares the supported service) + Device Name (the device name displayed in the App).

5.3 IRQ Event Callback

All BLE events are dispatched through a single IRQ callback: `CONNECT` / `DISCONNECT` manage the connection state, and `GATTS_WRITE` reads the data written by the central.

```
def _irq(self, event, data):
    if event == _IRQ_CENTRAL_CONNECT:
        self._conn_handle, _, _ = data
        print("BLE Connected")

    elif event == _IRQ_CENTRAL_DISCONNECT:
        self._conn_handle = None
        print("BLE Disconnected")
        self._pending_advertise = True               # re-advertise after
disconnect

    elif event == _IRQ_GATTS_WRITE:
        conn_handle, attr_handle = data
        if attr_handle == self._rx_handle:           # central wrote to the RX
characteristic
            buf = self._ble.gatts_read(attr_handle)  # read the written data
            print("Recv: %s" % bytes(buf).decode())
```

5.4 Sending Data

`gatts_notify(conn_handle, attr_handle, data)` pushes data to the connected central.

```
def send(self, data):
    if self._conn_handle is not None:
        self._ble.gatts_notify(self._conn_handle, self._tx_handle, data)
        return True
    return False
```

5.5 Main Loop

Every 500ms: check whether re-advertising is needed -> if connected, push data.

```
ble = BLESlave("ESP-IDF")
while True:
    ble.tick()                                       # check whether re-
advertising is needed
    if ble.is_connected():
        ble.send(b"Hello from ESP32")                # periodic push
    time.sleep_ms(500)
```

---

6. Operating Procedures

6.1 Dual-Board Debugging

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3.Thonny IDE`.

1. Power on the slave first and run `ble_slave.py`
2. Run `ble_master.py` on the central

6.2 Slave Serial Output Example

```
===== ESP32-S3 BLE Slave =====
[BLE] Init ...
[BLE] Init OK, TX=16 RX=19
Ready. Waiting for connection...
[BLE] Advertising as 'ESP-IDF' ...
==================================
BLE Connected
Sent: Hello from ESP32
Sent: Hello from ESP32
...
==================================
Recv: HELLO FROM MASTER
Sent: Hello from ESP32
...
```

6.3 Verification Methods

Operation	Expected Result
Run the slave first, then the central	The central automatically scans and connects
Power off the slave	The central prints `Disconnected` and rescans
After the central connects	The slave receives `Recv: HELLO FROM MASTER`

---

7. Common Problems

Symptom	Cause	Solution
The central cannot find the slave	The slave is not advertising or the name does not match	Confirm the slave is running and the `TARGET` variable matches the slave name
Disconnects immediately after connecting	Incompatible MTU/connection parameters	Usually normal — the ESP32 will automatically re-advertise
The slave writes data but the central has no response	IRQ callback not dispatched correctly	Confirm the `attr_handle == self._rx_handle` check
Notify has no data	CCCD not written correctly	The central needs to write the CCCD to enable Notify
Stuck at `[BLE] Init ...`	The BLE stack is not ready after `active(True)`	Check whether `time.sleep_ms(200)` is sufficient